



CS 498: Mixed Precision Training

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Model Parallelism:

- Tensor parallelism: communication-heavy
- Pipeline parallelism: tuning microbatch sizes to minimize bubbles
- Data parallelism: cannot run if layer (model) size exceeds GPU capacity

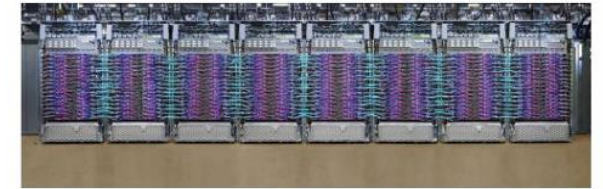
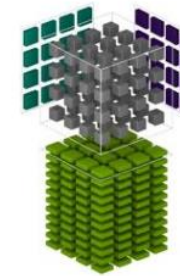
Minimizing compute and memory requirements is the key

- **Mixed Precision Hardware**
- **What is Mixed Precision Training?**
- **Considerations for Mixed Precision**
- **Mixed Precision Software**

Public
cloud



TensorCore



2006

2007

2016

2017

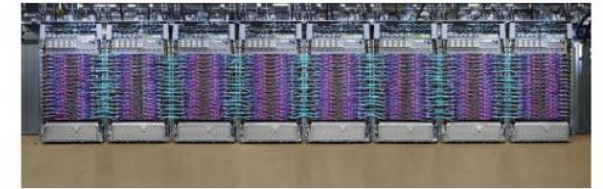
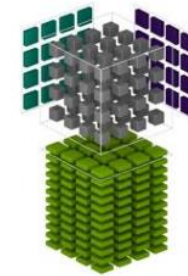
2019

Compute scaling

Public
cloud



TensorCore



2006

2007

2016

2017

2019

Compute scaling

Tensor Cores: Matrix Multiplication Units

- **Tensor cores are:**
 - Special hardware execution units
 - Execute matrix multiply operations in parallel
 - Fused multiply-add on small matrices in one instruction cycle
 - Built to accelerate deep learning



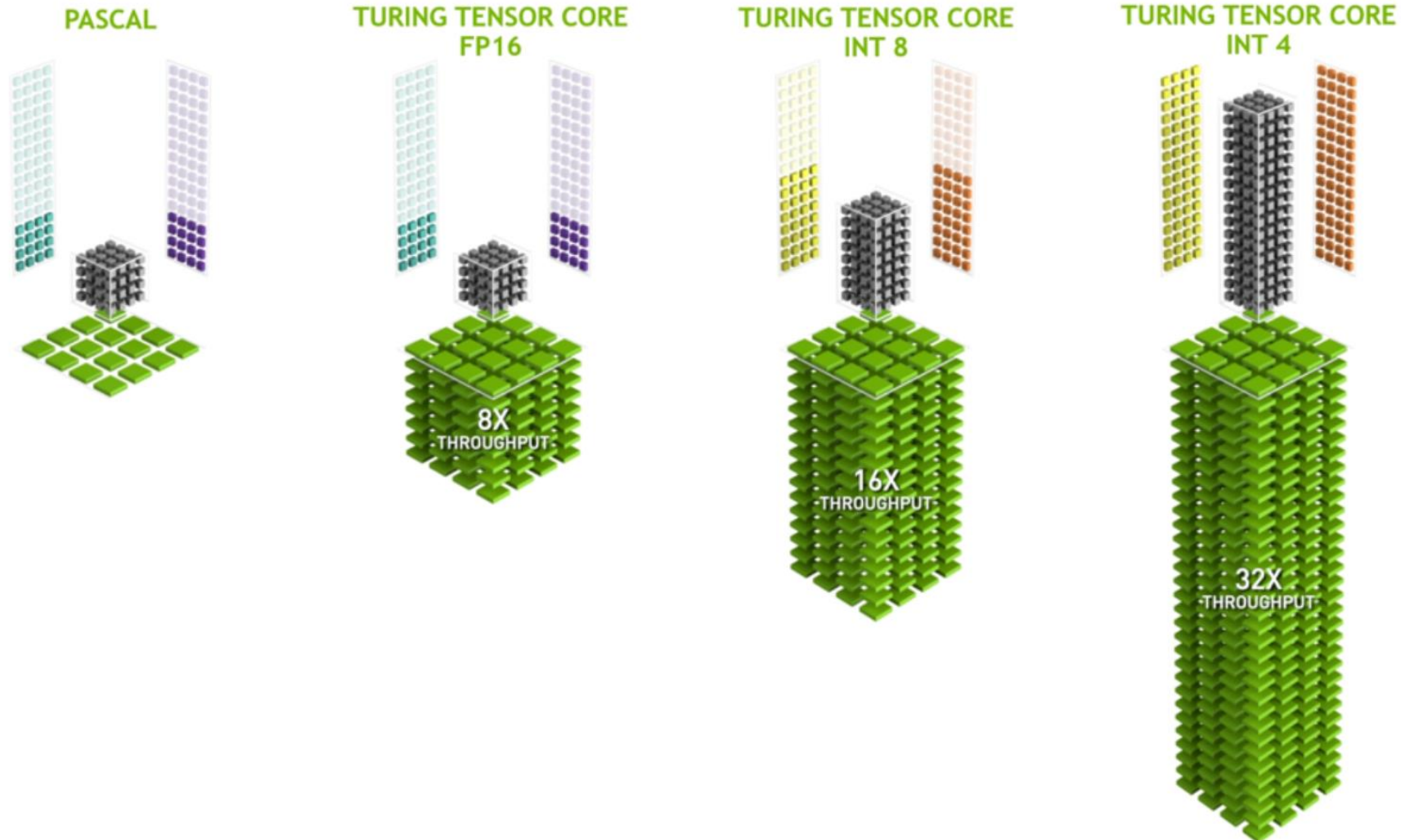
Tensor Cores: Matrix Multiplication Units



- **Tensor cores are:**
 - Special hardware execution units
 - Execute matrix multiply operations in parallel
 - Fused multiply-add on small matrices in one instruction cycle
 - Built to accelerate deep learning
- **Different flavors**
 - (V100) Volta Tensor Cores FP16
 - (T4) Turing Tensor Cores FP16/INT8/INT4
 - ...



Tensor Cores: Different Flavors



Tensor Cores: Mixed Precision Matrix Math on 4x4 Matrices



$$\mathbf{D} = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

$$\mathbf{D} = \mathbf{AB} + \mathbf{C}$$

What is Mixed Precision Training?

What is Mixed Precision Training? In a Nutshell



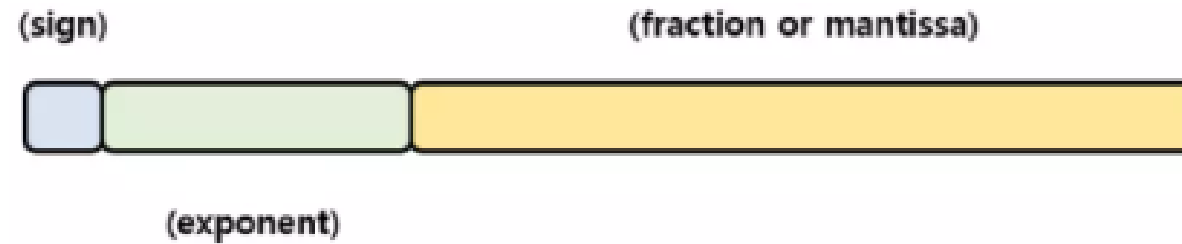
- Idea that you can train deep neural networks in multiple precisions:
 - Make precision decisions per layer or operation
 - Full precision (Fp32) where needed to *maintain task-specific accuracy*
 - Reduced precision (Fp16) everywhere else for speed and scale

What is Mixed Precision Training? In a Nutshell



- Idea that you can train deep neural networks in multiple precisions:
 - Make precision decisions per layer or operation
 - Full precision (Fp32) where needed to *maintain task-specific accuracy*
 - Reduced precision (Fp16) everywhere else for speed and scale
- By using *multiple* precisions, we can have the best of both worlds: **speed** and **accuracy**
- Goal: accelerate deep neural network training with mixed precision under the constraints of **matching accuracy of full precision training** and **no changes to how model is trained**

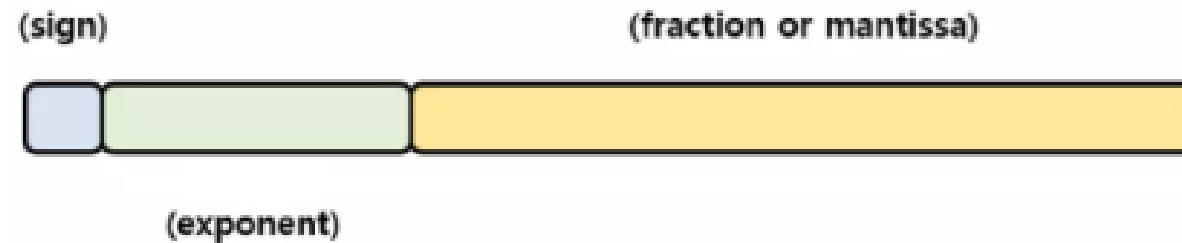
- Number (e.g., 2.45) can be represented by $(-1)^S * (1.M) * 2^{(E - Bias)}$



IEEE 754 Floating Point Representation Recap



- Number (e.g., 2.45) can be represented by $(-1)^S * (1.M) * 2^{(E - Bias)}$



Half Precision (Floating Point 16)	1 bit	5 bit	10 bit
Single Precision (Floating Point 32)	1 bit	8 bit	23 bit
Double Precision (Floating Point 64)	1 bit	11 bit	52 bit
Quadruple Precision (Floating Point 128)	1 bit	15 bit	113 bit

- **Accelerates speed**
 - Tensor cores are 8x faster than FP32

- **Accelerates speed**
 - Tensor cores are 8x faster than FP32
- **Reduces memory bandwidth and footprint pressure**
 - FP16 halves memory traffic compared to FP32
 - FP16 halves the size of activation and gradient tensors

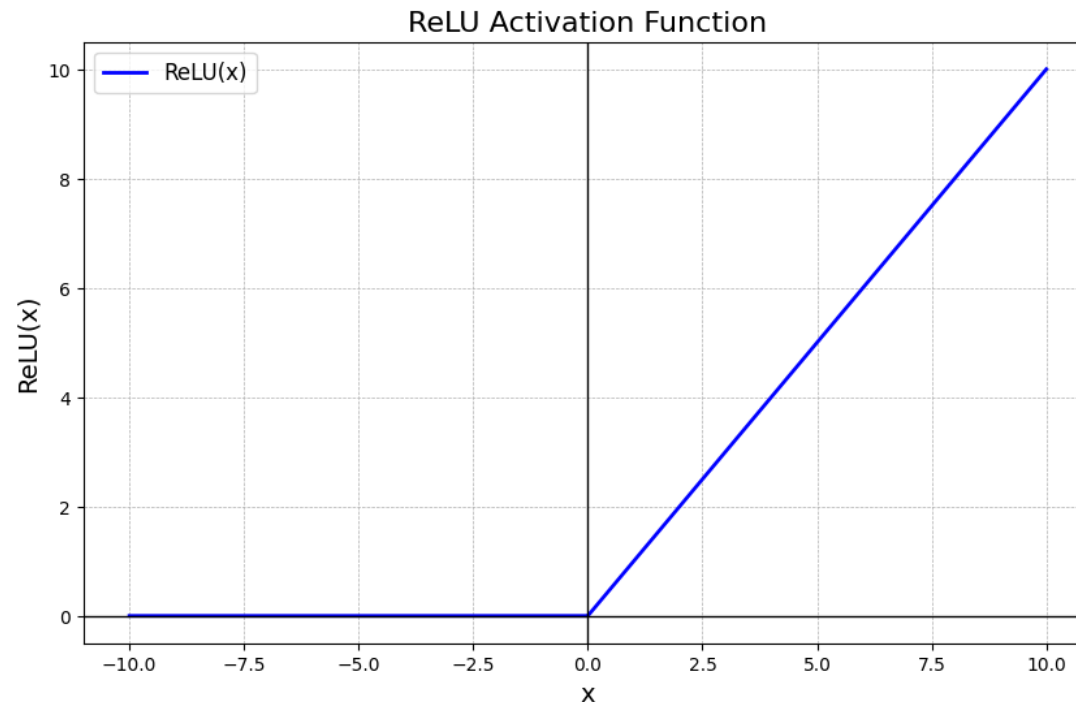
- Models cannot always converge properly in pure FP16
 - FP16 has a narrower dynamic range than FP32
 - May cause underflow/overflow issues and other arithmetic issues

- Models cannot always converge properly in pure FP16
 - FP16 has a narrower dynamic range than FP32
 - May cause underflow/overflow issues and other arithmetic issues
- **Weight updates**
 - Optimizer takes very **small increments** when search narrows into a solution
 - Late updates often cannot be represented in FP16, but can be crucial for accuracy

Considerations for Mixed Precision

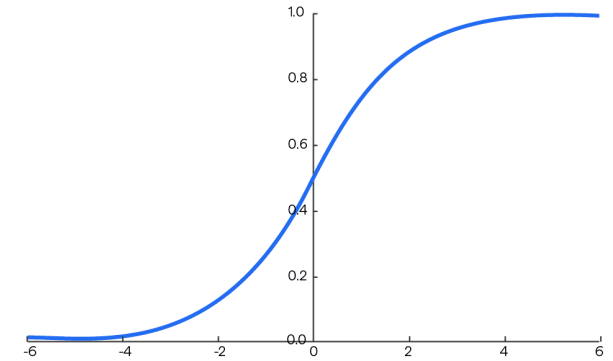
- Goal #1: Make mixed precision training general purpose, not only for limited class of applications
- Goal #2: With no changes to hyperparameters or the model architecture

- Operations that can use FP16
 - Matrix multiplications
 - Most element-wise operations (e.g., relu, tanh, add, sub)



- Operations that can use FP16
 - Matrix multiplications
 - Most element-wise operations (e.g., relu, tanh, add, sub)
- Operations that need FP32 mantissa
 - Reduction operations (e.g., softmax, layernorm)

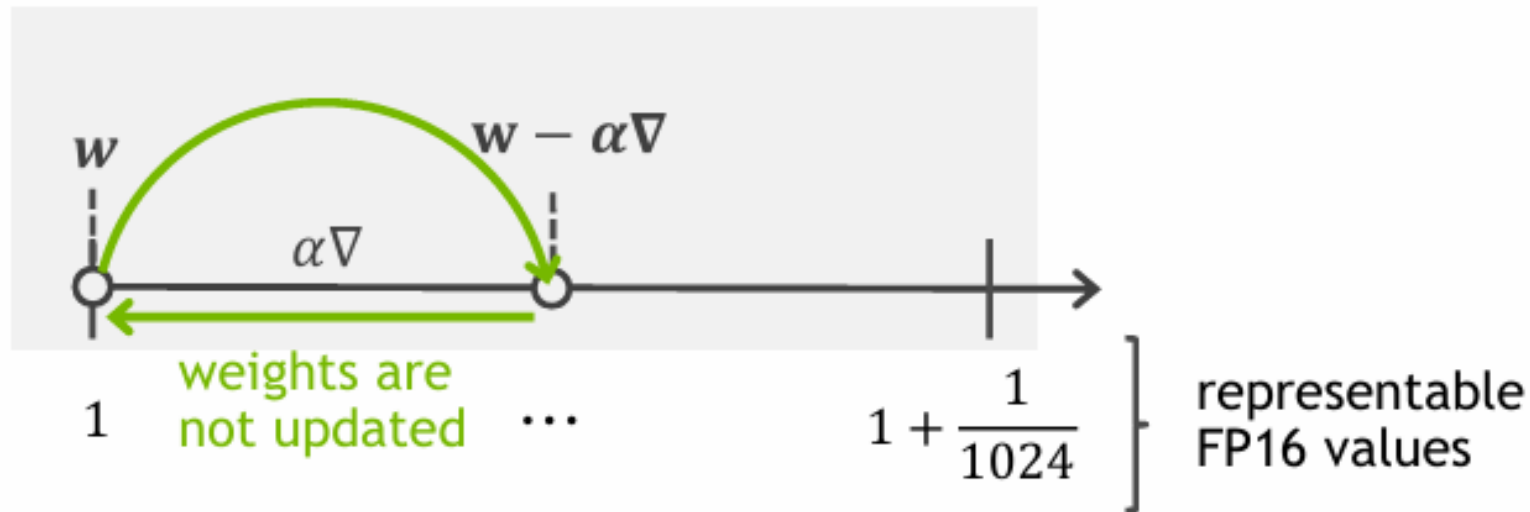
Softmax Function



- Operations that can use FP16
 - Matrix multiplications
 - Most element-wise operations (e.g., relu, tanh, add, sub)
- Operations that need FP32 mantissa
 - Reduction operations (e.g., softmax, layernorm)
- Operations that need FP32 range
 - Element-wise operations where $|f(x)| \gg |x|$, e.g., exp, log, pow
 - Loss functions

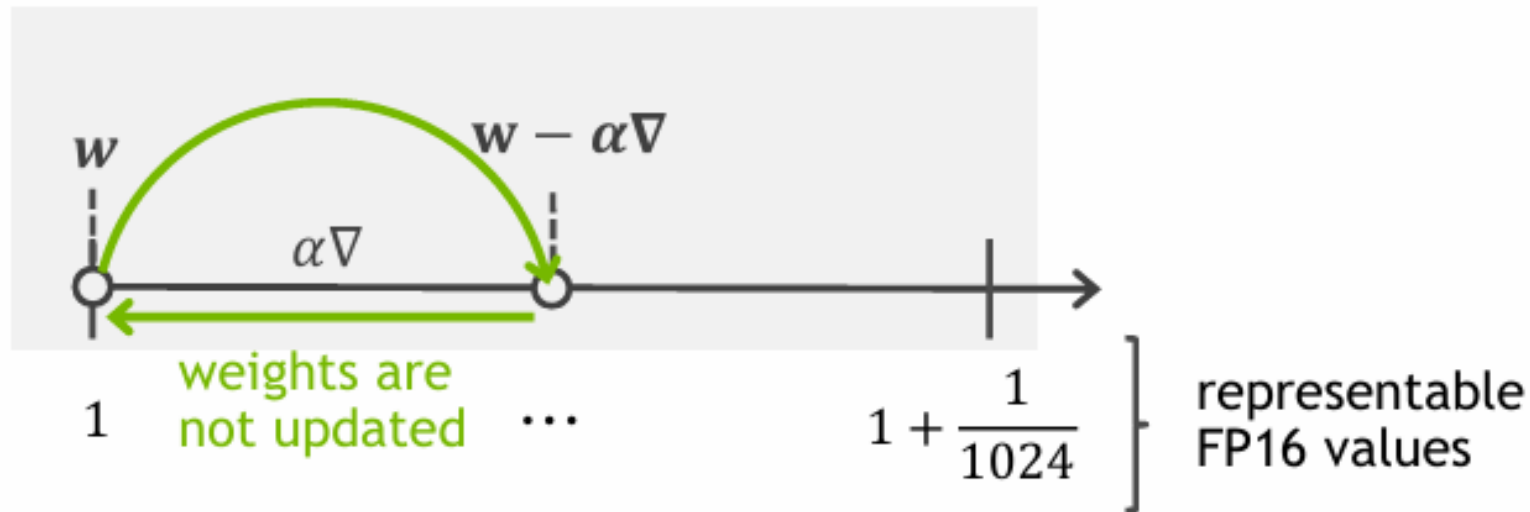
- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha \nabla$

- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha \nabla$



Example:
 $w \times 0.01$ leads to $\sim 2^7$ ratio,
 $w \times 0.001$ leads to $\sim 2^{10}$ ratio

- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha \nabla$



Example:
 $w \times 0.01$ leads to $\sim 2^7$ ratio,
 $w \times 0.001$ leads to $\sim 2^{10}$ ratio

- **Conservative solution:** keep *master copy* of weights and *optimizer state* in FP32 so small updates can accumulate

Master Weights: Illustration



2. Make an FP16 copy and forward/backward propagate in FP16



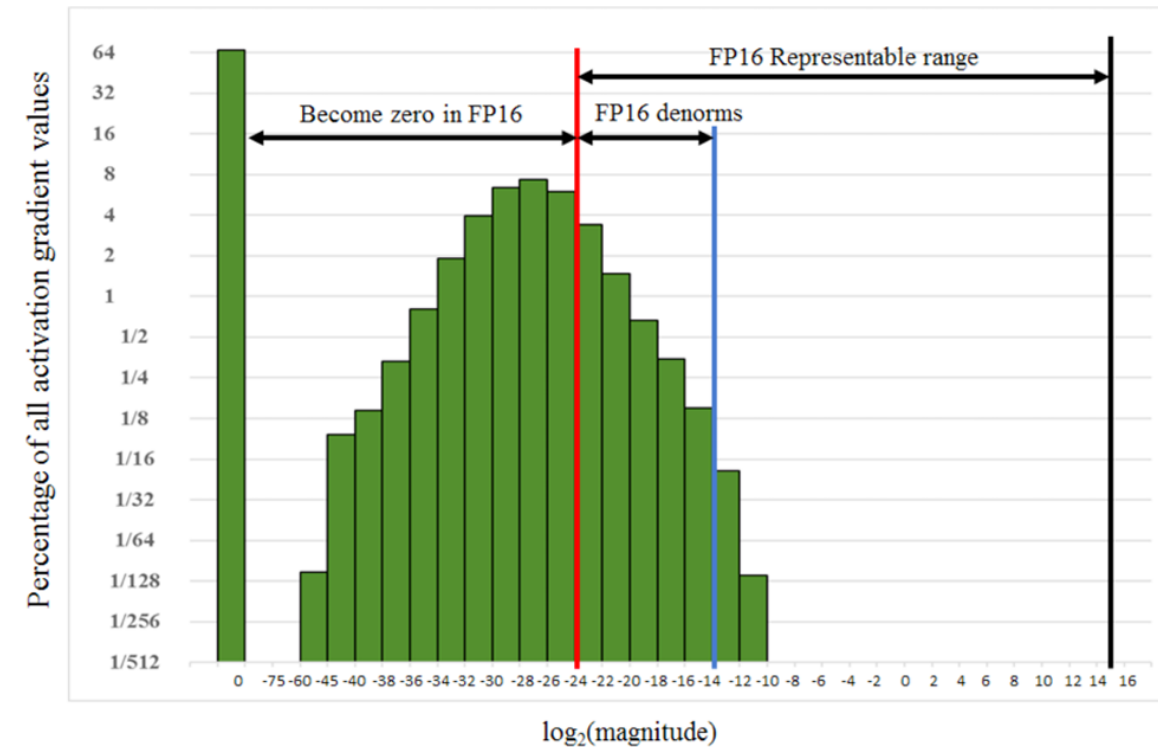
1. Keep weights in FP32

Master-Weights (F32)

Loss Scaling: Put All Tensors in FP16 Range



- Gradients are small:
 - If cast to Fp16, some lost to zero
 - Much of fp16 representable ranges was left unused
- Solution:



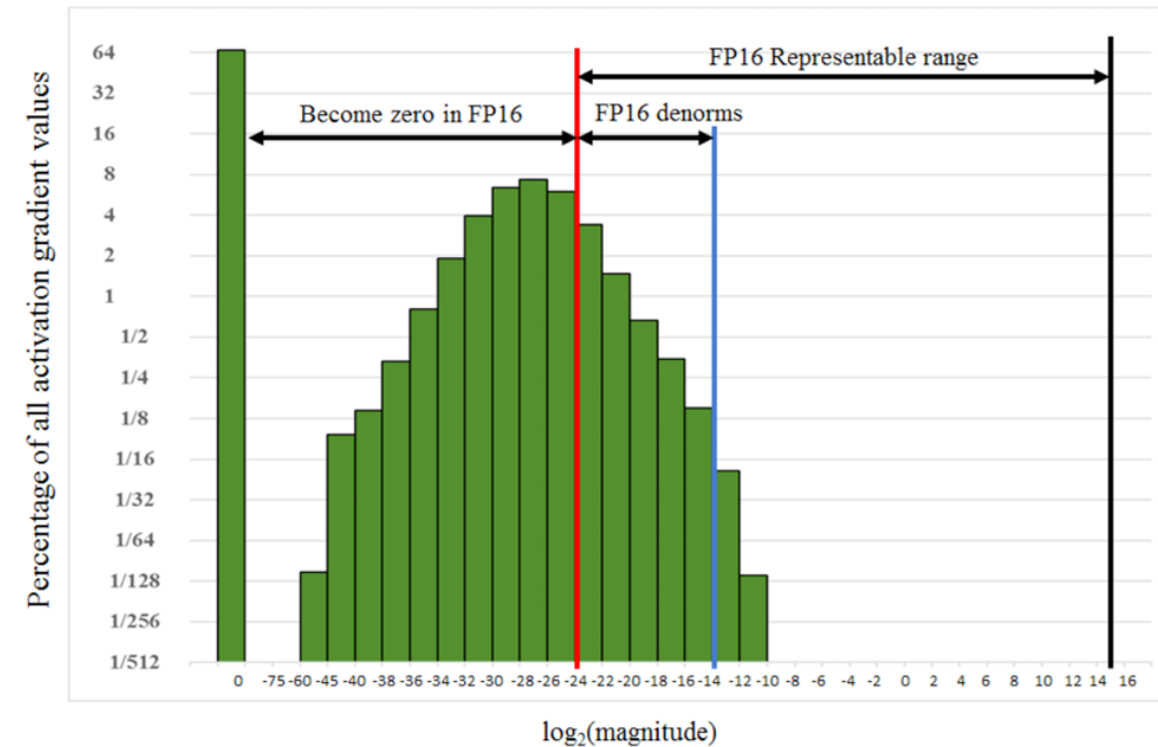
Histogram of activation gradient values

Loss Scaling: Put All Tensors in FP16 Range



- Gradients are small:
 - If cast to Fp16, some lost to zero
 - Much of fp16 representable ranges was left unused

Question: How can we make better use of FP16 representable range to capture gradients?

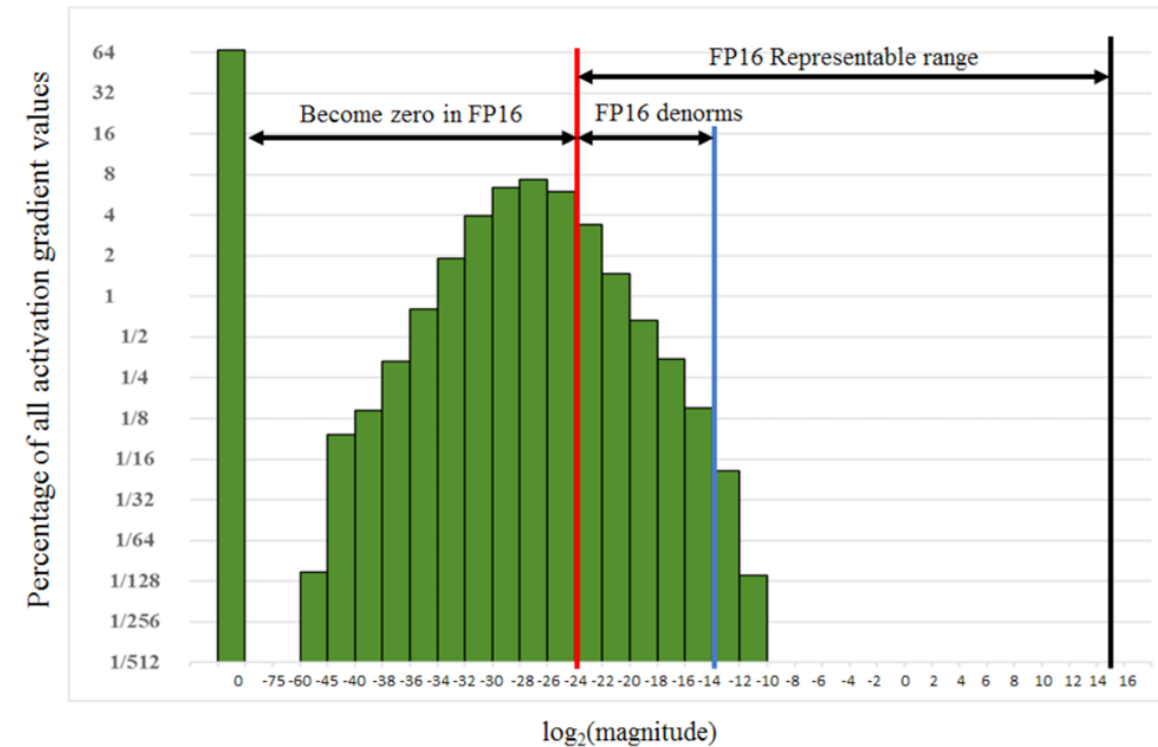


Histogram of activation gradient values

Loss Scaling: Put All Tensors in FP16 Range

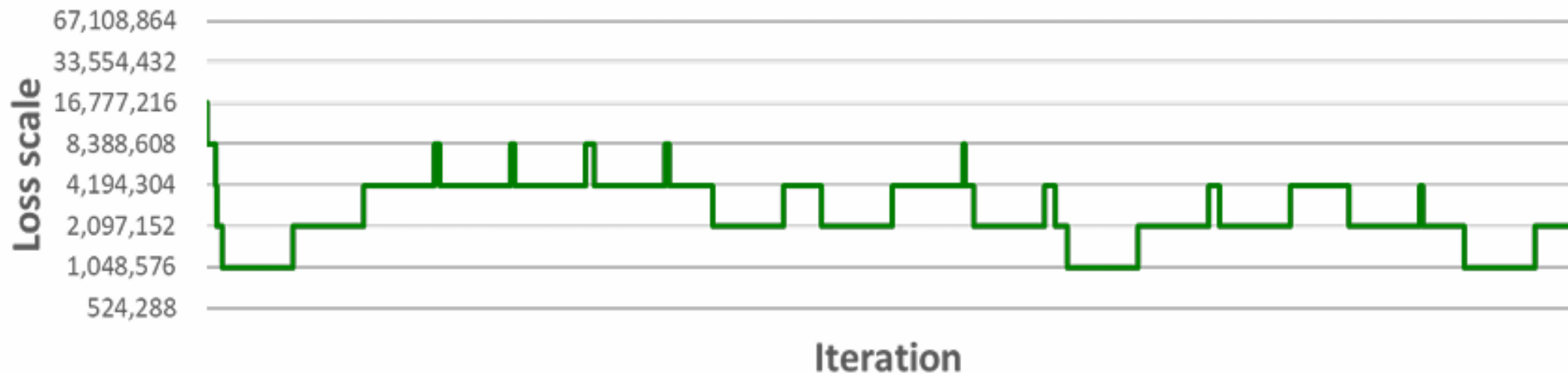


- Gradients are small:
 - If cast to Fp16, some lost to zero
 - Much of fp16 representable ranges was left unused
- Solution:
 - **Shift** small gradients values to fp16 range
 - Scaling gradients during backpropagation to prevent underflow
 - Gradients are scaled before backpropagation begins and rescaled before updating weights

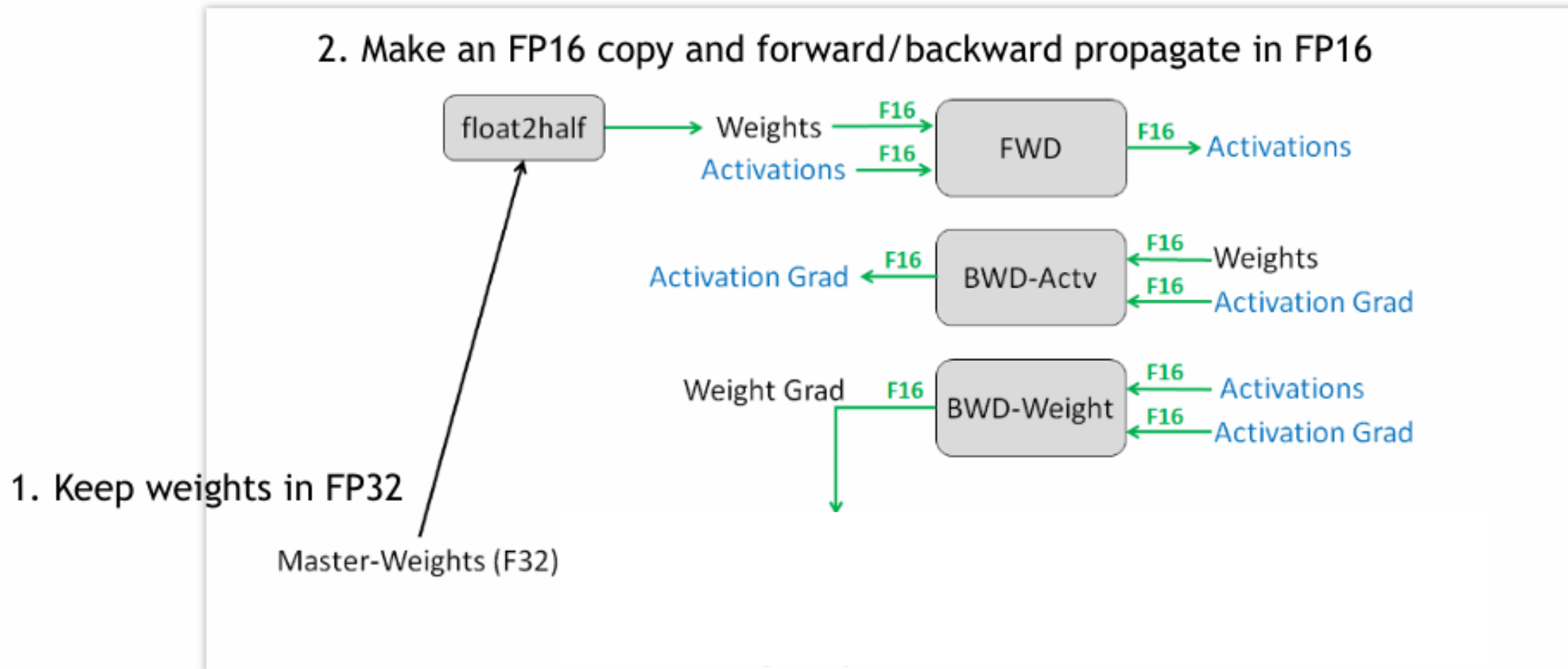


Histogram of activation gradient values

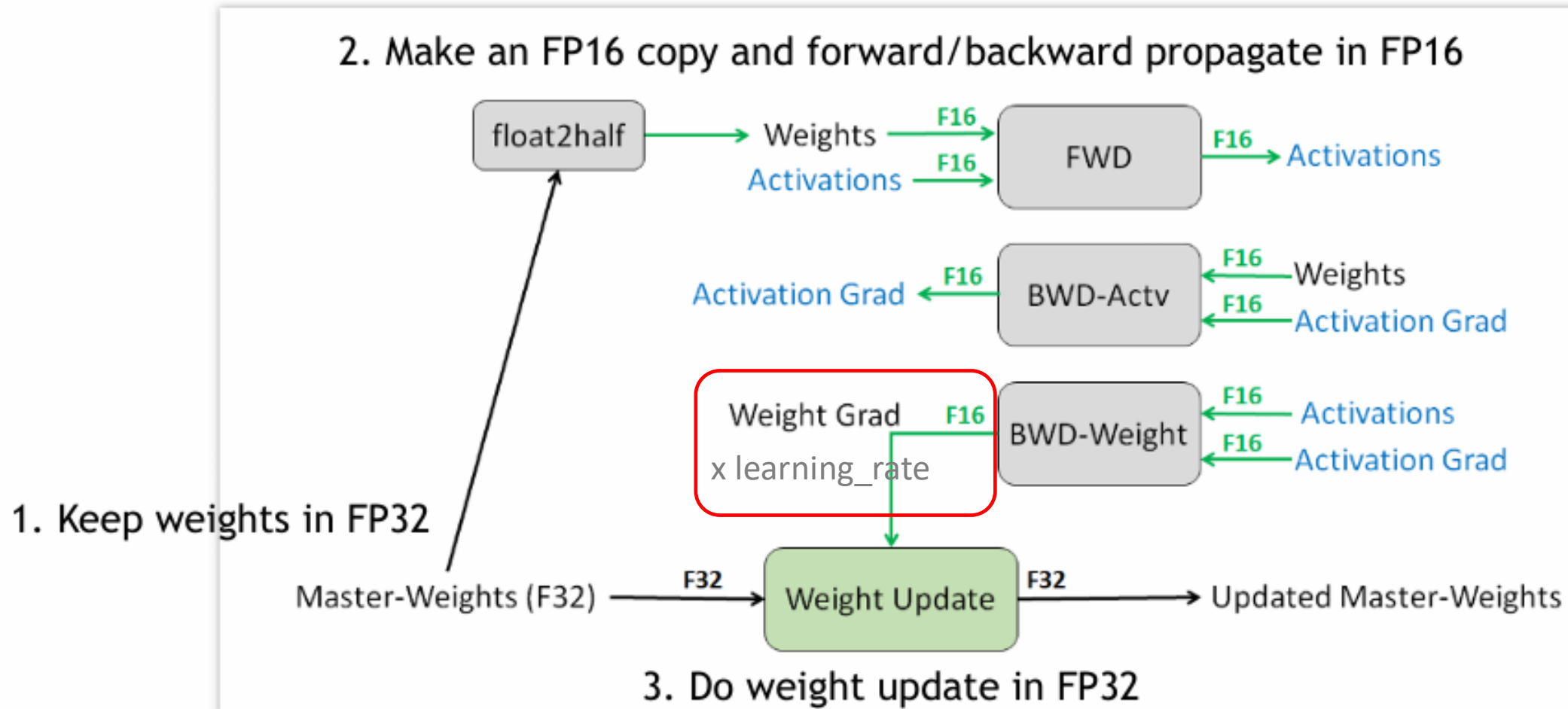
- 1. Start with very large scale factor (e.g., 2^{24})
- 2. If gradients overflow (with Inf or a Nan): decrease the scale by 2 and skip the update
- 3. If no overflows have occurred for some time: increase the scale by 2 (e.g., 2000 iterations)



Master Weights: Illustration



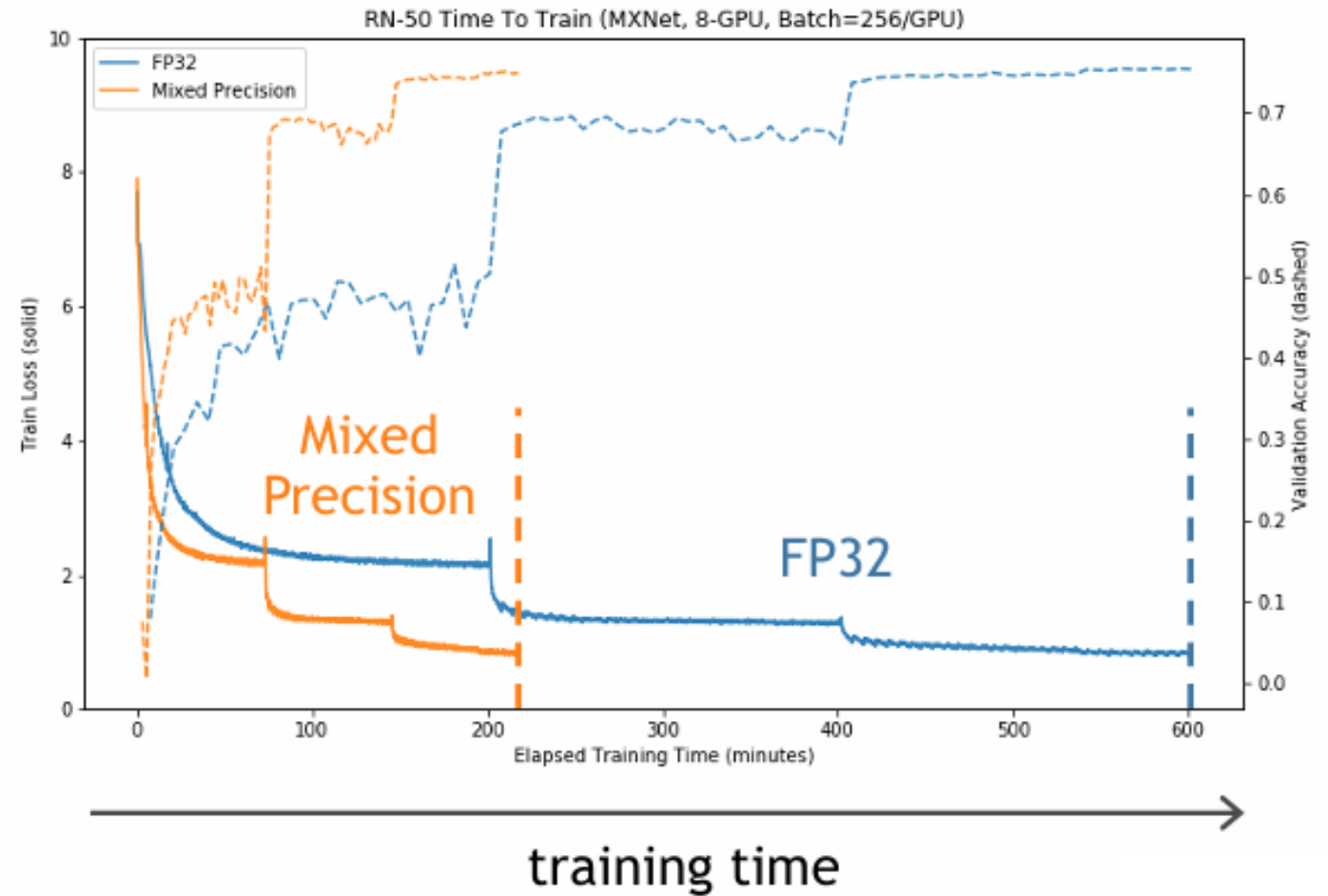
Master Weights: Illustration



Mixed Precision Training Example



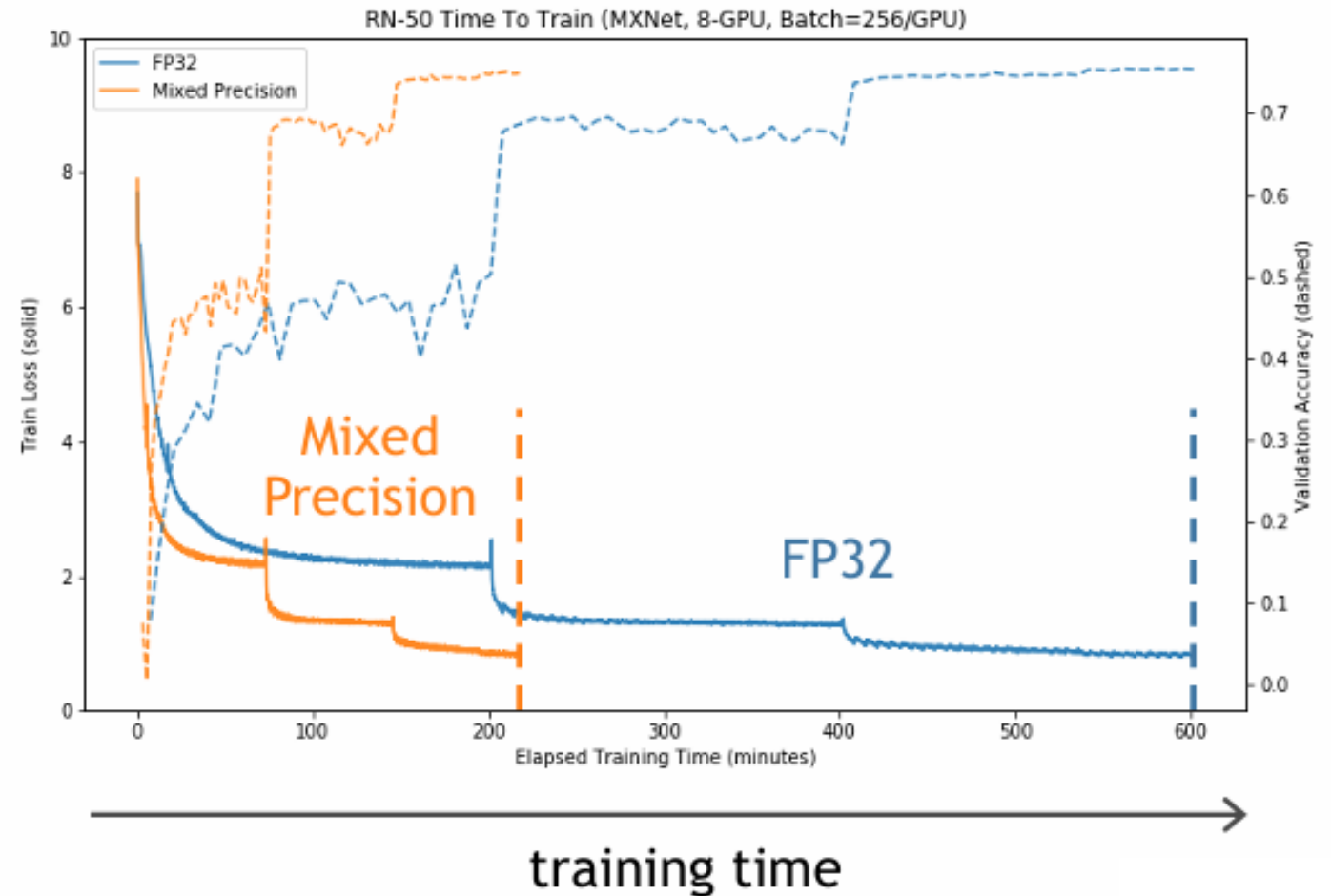
- ResNet-50 training for ImageNet classification
 - 8 GPUs on DGX-1



Mixed Precision Training Example



- ResNet-50 training for ImageNet classification
 - 8 GPUs on DGX-1
- Comparing to FP32 training
 - **~3x speedup**
 - **Equal accuracy**
 - No hyperparameters changed



- Available for major DL frameworks

PyTorch | DeepSpeed | Megatron | ...

- Works with all types of optimizers
 - SGD, Adam, etc
- Works with multiple models, optimizers, and losses

- **Mixed precision improves efficiency but may hurt quality**
 - FP16 vs. INT8
- **Automatic Scaling at runtime may help quality**
 - E.g., scaling the gradient to prevent underflow